Project Option 1: Memory-Aware Agent Edge Serving
Phase 1 Report: Baseline and Measurement

Ethan Gross (200135377), CSE 291A Fall 2026

1 Question, Setup, and Baseline

Question. On a memory-limited personal machine, how do concurrency, growing contexts and idle time between agent steps create memory pressure, and what limits completed agent work under a FIFO, fixed-concurrency baseline?

Setup. llama.cpp `llama-server`, pinned to release b11721 (commit `ff5a23376`), serving Qwen3-8B (GGUF Q4_K_M) with an fp16 KV cache (144 KiB per token), flash attention, and all layers solely on the GPU. I used my personal Windows 11 desktop with an AMD Ryzen 9 7900X, an NVIDIA RTX 5070 Ti, and 32 GB of DDR5 RAM. I disabled the GPU memory fallback to system RAM so running out of VRAM fails instead of silently slowing down. Model, precision and server flags were the same in every comparison.

Memory budget. All slots share one KV pool (`-kvu`) of 65,536 tokens (≈9 GiB, next to ≈4.8 GB of weights). The pool is allocated at startup, so GPU memory use is flat (≈14,100 MiB in every E2 run); what matters is how full the pool is. The server does not report this, so the runner tracks it from the tokens it sends and receives.

Baseline. A Python asyncio runner with:

- FIFO admission and K concurrent slots (server `-np` $= K$), so all queueing happens in the runner.
- Pinned slots: a task keeps its slot, and its cached context, across tool calls.
- Peak reservation: a task reserves its peak context when admitted; the queue head waits until it fits; a task larger than the pool is rejected (`ctx_overflow`).
- A finished task's slot is erased before its reservation is freed.

Prompts are exact token arrays, with temperature 0 and fixed output lengths. Failures are classified (e.g. `timeout`, `kv_exhausted`, `ctx_overflow`) and never retried. Every request, task and 200 ms system sample is logged, along with the commit, server command and file hashes. Every run is checked: completed + failed = planned tasks, and every prompt and output has its planned length.

Workloads. Defined in `configs/workloads/`; the same YAML and seed always produce the same tasks. Prompts are exact token slices of Project Gutenberg books [5]. All mixes use 16 closed-loop agents and a 512-token shared prefix.

Mix	Steps	Initial context	Output/step	Tool output	Tool delay
chat	1	256–1,024	64/128/256	–	–
long_context	1–2	6,144–12,288	128	256	1 s
long_output	1	512–2,048	512/768/1,024	–	–
agentic	4–8	1,024–2,048	128/256	256–1,024	lognormal, median 2 s
blend	40% chat, 10% long_output, 20% long_context, 30% agentic				
trace_agentic	5–12 [2]	Mooncake trace [1]	Mooncake (1–2,048)	lognormal, median 800 [3]	lognormal, median 0.23 s [2] (heavy: 0.3 s, σ 2.98 [3])

Workload Design. The hand-designed mixes cover what the course asks for: short and long contexts, short and long responses, and multi-step tasks with tool delays. The two trace mixes add realism. Public traces have token counts and timings but no text, so they set the distributions while the tokens still come from the corpus:

- Context and output lengths: Mooncake's tool-and-agent production trace [1].
- Step counts and trace_agentic's tool delays: SWE-bench and BFCL agent runs reported by Continuum [2].
- trace_agentic_heavy's long-tailed tool delays and the tool output size: TraceLab's coding-agent sessions [3].

The two trace mixes have identical tasks and differ only in tool delays. Details and limitations are in Appendix B.

2 Experiments (171 runs, all valid)

Each experiment answers one Phase 1 question from the course. E1 gives the cost model, E4 follows InferCept's question of whether to keep or drop KV during tool calls [4], and E7 was added after E2 to explain its slowdown. Each experiment was piloted before the full grid.

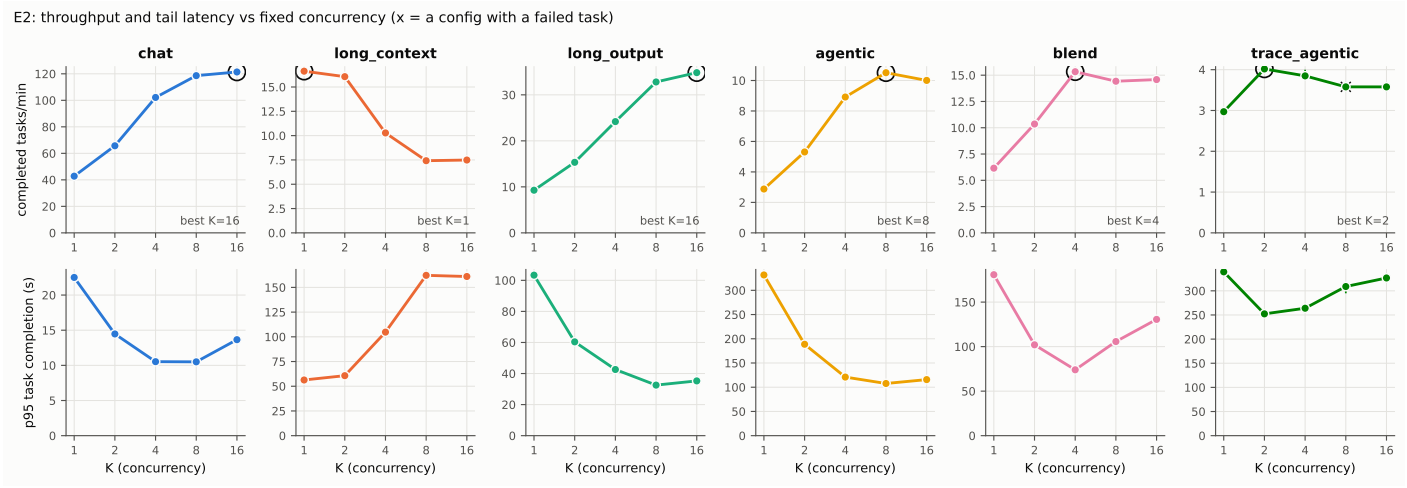
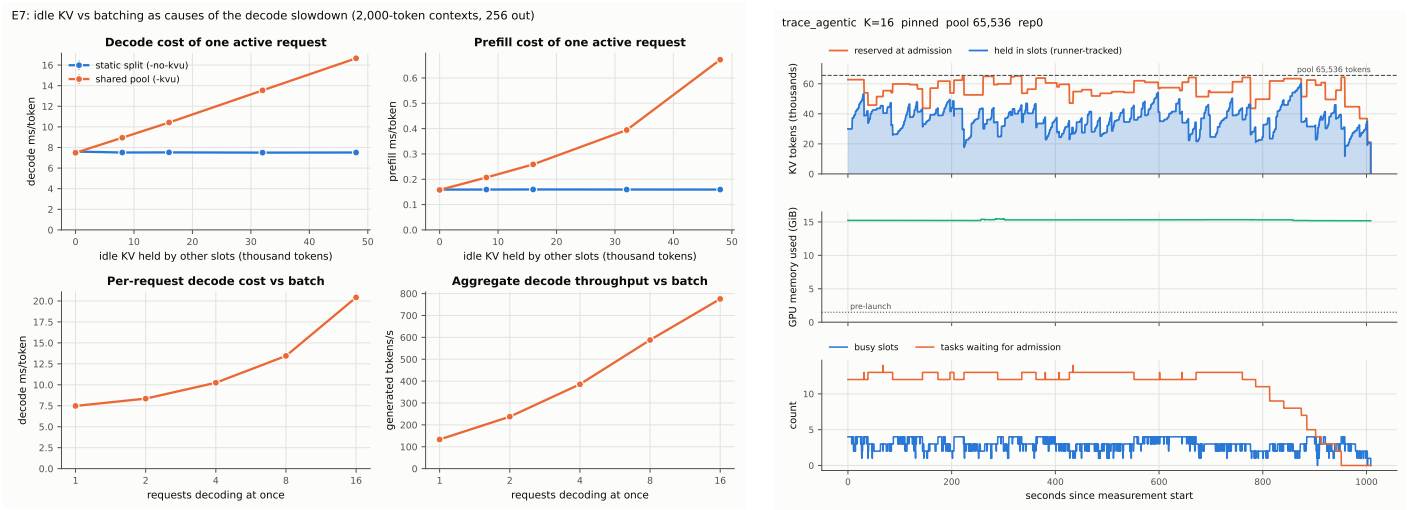


Figure 1: E2: completed tasks/min (top) and p95 task completion time (bottom) vs fixed concurrency K. Circle = best fixed K (highest throughput with no failed task; within 2%, lower p95).



(a) E7: idle KV vs batching. **(b)** trace_agentic, $K = 16$, one run.

Figure 2: The two causes of the bottleneck. (a) In the shared pool, idle KV in other slots slows the active request; with a static split it does not. (b) Reserved KV (orange) stays near the pool limit while used KV (blue) is about 36k; at most 4 slots are busy while about 12 tasks wait.

ID	What changes (everything else at baseline)	Reps
E1	Prompt 256–16,384 × output 32/128/512, $K = 1$ (cost model)	3
E2	$K \in \{1, 2, 4, 8, 16\}$ × six mixes; tasks per run = $\min(N, \max(20, 6K))$	3
E3	Context growth: 8 steps, initial context 1,024 vs 8,192, $K \in \{1, 4\}$	2
E4	Tool delay 0/1/2/5/15 s (agentic) and both trace mixes × slot mode pinned vs any, $K = 4$	3
E5	Shared prefix 0 vs 4,096 × prompt cache on/off × erase vs keep finished slots, $K = 4$	2
E6	Pool c = 65,536/32,768/24,576/16,384 × admission gate on/off, trace_agentic, $K = 4$	2
E7	Microbenchmark: one active request vs idle KV in other slots (shared vs static split); batch size 1–16	3

Points are medians over reps, with min–max bars. One task failed unexpectedly (a client connection timeout in E2, trace_agentic, $K = 8$); it is counted, and that setting is excluded when choosing the best K.

3 Results

Concurrency (Fig. 1). The best fixed K depends on the mix: 16 for chat (42.8→121.4 tasks/min) and long_output (9.3→34.8), 8 for agentic (2.9→10.5), 4 for blend (6.2→15.3), 2 for trace_agentic (3.0→4.0), and 1 for long_context. long_context gets *slower* with concurrency (16.6→7.4 tasks/min at $K = 8$; p95 56→162 s) because each request waits behind other slots' long prompts (decode 9→220 ms/token). At low K, most of a task's time is spent waiting for admission. These best-K values are the Phase 2 baseline.

Idle KV slows everyone (Fig. 2a). With 48k idle tokens in other slots, the active request decodes 2.2× slower (7.48→16.66 ms/token) and reads its prompt 4.3× slower (0.158→0.672 ms/token). With a static split, neither changes. That is about +0.19 ms per decoded token for every 1k idle tokens. Batching is cheaper: 16 requests at once each decode

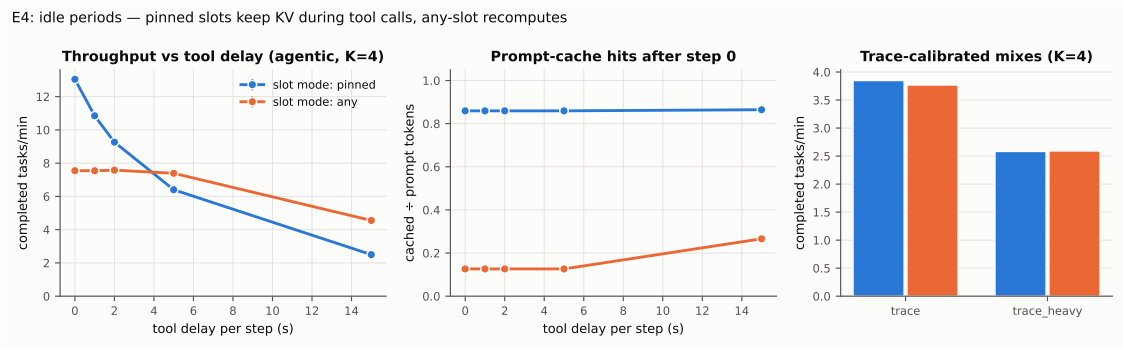


Figure 3: E4 ($K = 4$): pinned slots keep a task's cache across tool calls; *any* recomputes the prompt.

at 20.4 ms/token but produce 776 tokens/s in total, versus about 130 for one.

Peak reservations block admission (Fig. 2b). In trace_agentic at $K = 16$, about 56k tokens are reserved on average but only 36k are used, so at most 4 slots are ever busy. agentic shows the same gap (59k vs 37.5k). This is why trace_agentic stops improving after $K = 2$.

Context growth (E3). Going from $K = 1$ to 4 gives 2.3× throughput with 1k-token starting contexts but only 1.6× with 8k, as decode slows (9.6→16.1 ms/token) and used KV reaches 53k tokens.

Idle periods (Fig. 3). Pinned slots win when tool calls are short (13.0 vs 7.6 tasks/min at 0 s) because they recompute far less (114k vs 431k prompt tokens). Between 2 and 5 s the order flips (5 s: 6.4 vs 7.4; 15 s: 2.5 vs 4.6), because pinned slots hold their KV for the whole tool call. On the trace mixes the two are within 3%, so the slot choice matters little there.

Reuse (E5). The prompt cache matters most: turning it off cuts throughput by 26%, or by 53% with a 4,096-token shared prefix. Keeping finished slots helps only with a shared prefix (+14%).

Memory budget (E6). Without the admission gate, 4 of 24 tasks fail at 65,536 tokens and all 24 fail at every smaller pool, because one overflow fails every request in flight. With the gate, all 24 finish down to 32,768; at smaller pools only tasks larger than the pool are rejected (1 and 8 tasks). Throughput at 16,384 covers only the 16 smaller tasks, so it is not comparable.

4 Dominant Bottleneck

Completed work is limited by *how the KV pool is held*, not by GPU compute power. There are two main causes I found while running the experiments:

1. Reserving each task's peak leaves about 36% of reserved KV unused, so agentic mixes run well below K.
2. Agents waiting on tools keep their KV in the pool, which slows every active request (Fig. 2a). Pinning helps for short tool calls and hurts for long ones (Fig. 3).

Raising K isn't the solve-all solution. No single fixed K is right for all mixes.

5 Phase 2 Policy Directions

Both policies fit the existing scheduler policy interface. Each is tested alone and together.

- **P1: Usage-based admission.** Reserve what a task holds now plus its next step, not its peak, and admit while the free pool exceeds that need plus a headroom H. If no admitted task can continue, release the KV of the youngest idle task (P2's mechanism). Motivated by the 56k-reserved vs 36k-used gap (Fig. 2b). The risk is overflow, which fails every in-flight request (E6), so `kv_exhausted` is a primary metric and H is tuned on dev seeds.
- **P2: Idle-KV release.** When a tool call is expected to take longer than τ, erase the task's slot (`action=erase`) and recompute its prompt when it returns. Keeping costs about 0.19 ms/token per 1k idle tokens for every active request (E7); releasing costs one recompute at ≈5.5k tokens/s (E1). τ comes from this cost model and should land near E4's 2–5 s crossover. The expected delay uses only delays already observed, never the pre-generated value.

Evaluation. Compare against FIFO at the best fixed K per mix (chosen on dev seeds) and, for P2, against always-pinned and always-any. On held-out seeds, report completed tasks, p50/p95 completion time, throughput, peak KV, failures and policy overhead, with Poisson arrivals at several rates and changing mixes. Check that the largest tasks are not unfairly delayed. Expected low-benefit cases: chat (no idle time) and the trace mixes for P2. The validity checks ensure no gain comes from skipped tasks or shorter outputs.

Reproducibility. Sweeps: `python -m experiments.run_sweep configs/experiments/E<n>_*.yaml` (resumable); E7: `python -m experiments.e7_idle_kv`; analysis: `python -m analysis.metrics E<n>` and `python -m analysis.plots`. Runs used commit `66d6a71`; analysis code is at `36efa10`.

A Timeline and milestones

#	Date	Milestone	Status
1	2026-10-29	Stack pinned (llama.cpp b11721, Qwen3-8B Q4_K_M); runtime probes; VRAM vs pool size; shared-pool memory model agreed with TA	done
2	2026-11-01	Deterministic workload generator, client, FIFO scheduler with policy interface, samplers, validity checks	done
3	2026-11-01	Pool fixed at 65,536 tokens; pilots of all experiments valid; resumable sweep runner	done
4	2026-11-02	Full E1–E6 (171 runs) and E7	done
5	2026-11-03	metrics and figures from raw logs	done
6	2026-11-04	Phase 1 report submitted	done

B Workload sources and calibration

Pinned trace files, hashes and the fetch script are in `configs/trace_manifest.json` and `scripts/fetch_traces.py`; the full rationale is in `docs/workload_sources.md`.

Source	What it provides	Used for
Mooncake tool & agent trace [1] (Kimi, FAST'25; commit `3cca71d`, Apache-2.0)	23,608 production requests with input and output token counts (input median 6,346, p90 16,805; output median 30, p90 507).	Empirical draws of initial context (clipped to 256–16,384) and output per step (1–2,048) in both trace mixes.
Continuum [2]	GPT-5 agent runs: SWE-bench 10.9±2.1 turns, tool time 925±3,550 ms; BFCL web search 6.3±2.3 turns.	Steps uniform 5–12; trace_agentic tool delay lognormal (median 0.233 s, σ 1.66, clipped at 60 s).
TraceLab [3]	4,265 Claude Code and Codex sessions: tool call median 0.3 s, p90 13.6 s; median append ≈857 tokens per step.	trace_agentic_heavy tool delay lognormal (σ 2.98, clipped at 120 s); tool output lognormal (median 800, 16–8,192).
InferCept [4]	Keep / discard / swap KV during tool calls.	Framing of E4 and Phase 2 P2 (not a workload source).
Project Gutenberg [5]	Six public-domain books (1,674,729 tokens with the pinned tokenizer).	Token content of every prompt and tool output, as exact token slices.

Limitations. The trace mixes combine marginal distributions from three different systems; step count, context, output and delay are drawn independently. Every task shares the same 512-token prefix, while real prefix sharing is partial. Tool delays are clipped at 60 and 120 s, which trims the heavy tail that makes idle retention expensive. Outputs have forced lengths, so content and quality are not modelled (intended, for reproducibility). Mooncake's arrival timestamps were not used; candidate sources for Phase 2's changing arrival rates are the Azure LLM inference trace and BurstGPT.

References

[1] R. Qin et al. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. FAST 2025. Trace: `https://github.com/kvcache-ai/Mooncake/blob/main/FAST25-release/README.md`.

[2] H. Li et al. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230. `https://arxiv.org/abs/2511.02230`.

[3] TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560. `https://github.com/uw-syfi/TraceLab`.

[4] R. Abhyankar et al. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. ICML 2024.

[5] Project Gutenberg. `https://www.gutenberg.org` (book ids 1342, 2701, 98, 84, 1661, 2600).